

Architecture élite — v2 (consolidée)

Ce qui change vs la version précédente

Ajouté	Rejeté / phasé
Contrats de données formalisés (Pydantic)	Moteur de priorisation multi-enseignants
Knowledge Compiler (étape déterministe)	Corpus vectoriel "dédoublé" (reste 1 seule table)
Critique à deux couches (structurel → pédagogique)	—
Versioning des skills critiques	—
<code>priorite</code> (champ simple) + <code>contexte_classe</code> (optionnel)	Moteur de contexte complexe

0. Vision

Une plateforme, pas un générateur de fiches. Base de connaissances unique comme source de tout contenu généré. Agents autonomes sur le **comment**, jamais sur le **quoi**. Production à la demande, à l'unité ou en volume.

1. Principes directeurs

1. **Le fait ne se génère jamais** — programme, prérequis, progression viennent toujours de données structurées.
2. **La séquence est l'objet central**, pas la fiche — et une séquence peut couvrir plusieurs notions (`notion_ids[]`).
3. **Complexité proportionnelle à la preuve** — rien n'est construit avant qu'un besoin réel ne l'ait justifié.

4. **Autonomie bornée** — chaque agent a un périmètre explicite ; il ne peut pas en sortir.
 5. **Contrats explicites entre étapes** — chaque objet échangé entre agents est un schéma validé, jamais un dict libre.
-

2. Base de connaissances

Une seule table vectorielle (`corpus_vectoriel`), deux façons d'y accéder — pas deux systèmes séparés :

```
corpus_vectoriel(  
    id, embedding, contenu,  
    type          TEXT,          -- 'exemple_pedagogique' | 'fiche_reference'  
    niveau        TEXT,  
    chapitre      TEXT,  
    statut        TEXT,          -- 'brouillon' | 'validee'  
    exemplaire    BOOLEAN        -- marquée comme référence de style par un humain  
)
```

- `retrieve_pedagogical_examples(notion, k)` → **recherche sémantique large** sur `type='exemple_pedagogique'`
- `retrieve_style_reference(niveau, chapitre, k=3)` → **filtre strict** sur `type='fiche_reference' AND statut='validee' AND exemplaire=true AND niveau=X AND chapitre=Y`, puis tri par récence

Ça évite la dérive de style : seules les fiches explicitement marquées exemplaires nourrissent le style, pas toutes les fiches produites.

Plus : référentiel structurel (`notion` , `prerequis` , `progression`) et templates versionnés (`fiche_template_v1.json` , inchangé).

3. Contrats de données

Chaque objet échangé entre étapes est un schéma validé — plus de dict libre entre agents.

```
class CurriculumSpec(BaseModel):  
    notion_id: str  
    nom: str
```

```

    competences: list[str]
    objectifs: list[str]
    prerequis_ids: list[str]

class StyleReference(BaseModel):
    fiche_id: str
    extrait: str
    niveau: str
    chapitre: str

class GenerationContext(BaseModel):
    """Produit une fois par le Knowledge Compiler, figé et rejouable."""
    sequence_id: str
    notions: list[CurriculumSpec]
    exemples_pedagogiques: list[dict]
    references_style: list[StyleReference]
    regles: dict
    contexte_classe: Optional[dict] = None
    template_version: str
    curriculum_version: str

class SectionContent(BaseModel):
    section_id: str
    contenu: str
    methode: Optional[str] = None # colonne "Méthode et procédé"

class ValidationResult(BaseModel):
    structurel_pass: bool
    structurel_raisons: list[str]
    pedagogique_pass: Optional[bool] = None
    pedagogique_raisons: Optional[list[str]] = None
    section_a_regenerer: Optional[str] = None

class RenderedDocument(BaseModel):
    livrable_id: str
    sequence_id: str
    format: str
    contenu_final: dict
    valide: bool

```

GenerationContext étant figé, une génération peut être **rejouée exactement** sans redépendre de l'état live de la base — utile pour debug et pour comparer deux versions de skill sur le même contexte.

4. Pipeline complet

Demande (fiche unique OU batch : "génère le chapitre X")

|

▼

Agent Planificateur

- resolve_batch_plan()
- tri : progression.semaine ASC, priorite DESC ← champ simple, pas un ordonna
- check_prerequisites_covered() par séquence

|

▼ (file de Sequence – parallélisable)

Knowledge Compiler ← déterministe, PAS un agent (pas de décision LLM)

- fetch_curriculum_spec()
- fetch_prerequisites()
- retrieve_pedagogical_examples()
- retrieve_style_reference()
- charge regles(niveau) + contexte_classe si renseigné
- GenerationContext (figé, versionné, rejouable)

|

▼

Agent Rédacteur ← reçoit un contexte déjà prêt, ne fait plus de retrieval lui-m

- generate_section_pair_v{n}(section, GenerationContext) → SectionContent[]

|

▼

Agent Critique – deux couches

1. validate_structuel() Python, rapide, toujours exécuté
sections présentes / format / champs obligatoires / durées cohérentes
→ FAIL : retour ciblé au Rédacteur, AUCUN appel LLM
2. validate_pedagogique() LLM, SEULEMENT si (1) PASS
clarté / cohérence de progression / pertinence des exemples
→ FAIL : retour ciblé au Rédacteur
→ PASS : ValidationResult(valide=true)

|

▼

Agent Superviseur

- render_fiche() → export_render() → commit_batch()
- retries bornés (max N par section), escalade humaine si échec persistant

Le Critique en deux couches économise des appels LLM : la majorité des échecs (section manquante, template mal rempli) sont attrapés en Python avant tout coût de génération de jugement.

5. Agents — périmètre d'autonomie

Agent	Autonome sur	Jamais autonome sur
Planificateur	décomposition du batch, ordonnancement simple	ordre du programme, existence des prérequis
Rédacteur	formulation, choix des exemples (dans le contexte fourni), style des questions	objectifs, notions à couvrir
Critique	décision pass/fail, ciblage de la régénération	les critères eux-mêmes (fixes, pas inventés)
Superviseur	gestion des retries, escalade	tout le reste — coordination pure

Knowledge Compiler n'est pas listé comme agent : c'est une étape 100% déterministe (agrégation de skills existants), sans décision autonome.

6. Skills — versionnées pour les plus critiques

```
generate_section_pair_v1() / v2()
validate_pedagogique_v1() / v2()
```

Chaque livrable stocke le skill_version utilisé → comparaison A/B possible sans casser les générations déjà commises et signées.

Reste du catalogue de skills : inchangé par rapport à la version précédente

```
( fetch_curriculum_spec , check_prerequisites_covered , render_fiche , export_render ,
  commit_batch ... ).
```

7. Modèle de données

```
sequence (
  id, notion_ids[], semaine, statut,
  priorite          INT DEFAULT 0,          -- simple, PAS un moteur de scheduling
  contexte_classe   JSONB DEFAULT NULL,    -- optionnel : {effectif, materiel, d
  template_version, curriculum_version
)

generation_context(id, sequence_id, payload_json, compiled_at) -- objet figé, re
```

```
livrable(id, sequence_id, type, contenu_json, format, valide,
        skill_version, agent_trace_id)

validation_result(id, livrable_id,
                 structurel_pass, structurel_raisons[],
                 pedagogique_pass, pedagogique_raisons[],
                 section_a_regenerer)

agent_run(id, sequence_id, agent, input, output, decision, timestamp)
```

`priorite` et `contexte_classe` sont des champs, pas des sous-systèmes : triés/lus simplement, jamais bloquants s'ils sont absents.

8. Ce qui reste explicitement hors scope

- **Moteur de priorisation avec disponibilité enseignants** — aucun besoin observé de multi-profs/multi-classes en parallèle. Un tri `priorite DESC` suffit. À réévaluer seulement si ce cas d'usage apparaît réellement.
 - **Moteur de contexte de classe élaboré** — `contexte_classe` reste un JSON optionnel libre, pas un système de règles séparé, tant qu'aucune fiche réelle n'a montré que le contenu doit varier selon l'effectif/le matériel.
-

9. Stack

- **Agents** : LangGraph (graphe explicite, autonomie bornée par nœud)
 - **Contrats** : Pydantic
 - **Données** : Postgres + pgvector
 - **Queue** : Redis + RQ/Celery (mode batch)
 - **Templates** : Jinja2 + `fiche_template_v1.json`
 - **API** : FastAPI
-

10. Roadmap

Phase	Contenu
-------	---------

1 — MVP agentique complet	4 agents + Knowledge Compiler + Critique 2 couches + contrats Pydantic + <code>priorite / contexte_classe</code> simples. Tout ce qui est dans ce document.
2 — Volume	Parallélisation batch, cache, comparaison A/B des skills versionnées
3 — Style affiné	Corpus <code>fiche_reference</code> enrichi par l'usage réel, retrieval calibré
4 — Boucle d'apprentissage	<code>agent_run</code> exploité pour ajuster les skills selon les taux de validation réels

La phase 1 est plus riche que dans la version précédente — Knowledge Compiler, contrats et Critique à deux couches y entrent dès le départ parce que leur coût d'implémentation est faible et leur valeur immédiate. Le moteur de priorisation reste hors scope tant qu'aucun besoin concret ne l'impose.